

Trust Is a Spectrum, Not a Switch

A follow-up to "How Do I Prove I Am Who I Am?"

After I published my last piece on identity verification, a technically sharp reader pushed back on several of my assumptions. The pushback was good. It forced me to think harder, and this article is the longer answer I promised.

There were four challenges worth taking seriously.

Challenge 1: Human identity verification doesn't scale — and shouldn't

The first challenge was the most fundamental: applying human metrics to a machine world is the wrong framing from the start. Between you and the server you're trying to reach, there can be hundreds or thousands of intermediate hops — machines authenticating with machines, services calling services, agents orchestrating agents. Requiring human verification at each step doesn't just create friction. It creates a security catastrophe. The number of machines is growing non-linearly. The verification problem grows with it.

This is exactly right, and I want to say it clearly: **human identity verification belongs at the edge — one time, at the moment a person first asserts their presence to their own devices. Everywhere else, it must be machine identity, all the way down.**

The question I was really asking in my original piece — "how do I prove I am who I am?" — is a question about that initial assertion. How does a human being establish a root of trust that machines can then carry forward and verify without ever calling the human back? That's the hard problem. Everything after it is cryptographic delegation — and cryptographic delegation scales.

Challenge 2: PKI just shifts the problem — sovereign risk is real

The second challenge was more specific to my proposed solution: PKI expands the attack surface rather than shrinking it. Who manages the certificates? A central Certificate Authority is a single point of compromise. If the CA is breached, corrupted, or coerced, every certificate it issued becomes suspect. We've seen this: DigiNotar in 2011, Symantec's gradual distrust, government CAs issuing fraudulent certificates for interception. The instinct to centralise trust management consistently produces the exact thing we're trying to avoid.

This is correct. And it's why I'm not proposing a central CA.

The model I'm building toward is based on a different observation: **trust doesn't require a central authority. It requires multiple independent witnesses.**

When I publish my public key in five places — a DNS TXT record on my domain, a page on my company website, my GitHub profile, a signed file in a public repository, and a verification endpoint that anyone can query — no single entity controls my identity. If an attacker wants to impersonate me, they don't need to compromise one trusted third party. They need to simultaneously compromise my DNS registrar, my web

hosting, my GitHub account, and my repository. That's not impossible, but it's a fundamentally different threat model than "compromise the CA, compromise everything."

This is what I mean by a web of trust. Not a hierarchy with a root that can be cut. A mesh, where each node independently confirms the same claim.

Challenge 3: Your "reputable" entities become the next single point of failure

The third challenge pressed harder on the same point: Microsoft, CrowdStrike, and OnePassword have all, at various points, been the trusted supplier who caused the breach. If I build my identity model on "reputable organisations publish and maintain employee public keys," I'm describing a federated certificate authority by another name. And we know how that ends.

This lands. The CrowdStrike incident in particular is an argument against any architecture where one upstream supplier's failure cascades unconditionally into every downstream system that trusted them.

But I think there's a path through this, and it comes from distinguishing **trust breadth** from **trust depth**.

When I say "reputable organisations publish employee public keys," I'm not proposing that you trust those keys absolutely, or that you trust any single organisation as your only source of truth. I'm proposing that multiple organisations, independently publishing verifiable attestations, create a cumulative weight of evidence that no single supplier's failure can entirely collapse.

If my employer publishes my public key, and I also publish it myself, and a community keyserver cross-signs it, and a friend who knows me in real life has signed it — then a breach at my employer reduces the cumulative evidence but doesn't eliminate it. The remaining strands of the web still hold. You have a signal that the employer's attestation may be compromised, but the other attestations are still valid.

This isn't a solved problem. Key management for decentralised trust is genuinely hard. But the direction is right: **the answer to sovereign risk is not a better sovereign — it's the elimination of single sovereignty entirely.**

Challenge 4: Trust as a spectrum — and what that actually means

The fourth challenge was implicit in all of the above: trust is not binary. You can do everything right and still have a problem because someone in your trust chain did something wrong. There is no "fully trusted" state. There is only "trusted enough, for this purpose, with this level of verification, in this context."

This is the frame I want to build on, because I think it's the most generative way to think about the whole problem.

Trust is a spectrum, not a switch.

Consider how trust actually works in practice. When a stranger sends me a LinkedIn message, I give it very low trust — maybe enough to read it, not enough to click the link. When a colleague sends me the same message from a known email address, I give it higher trust — enough to open the attachment. When the same colleague follows up on a phone call we just had with a message confirming what we discussed, I give it high trust — enough to act on it.

None of these are "trusted" or "untrusted." They're different points on a spectrum, based on accumulated evidence across multiple channels. What makes the trust level adequate is the combination of signals, each independently verifiable.

The same logic applies to machine identity. An agent presenting a signed request from a key I've seen before gets moderate trust. An agent presenting a signed request from a key I've seen before, where the signature chain traces back to an entity I've independently verified through multiple channels, and whose behaviour history is consistent with past interactions — gets high trust. An agent presenting an unsigned request, from an unknown source, asking for elevated privileges — gets zero trust.

The question is never "is this trusted?" It's "is this trusted enough, for what we're about to do, given the evidence we have?"

What this means for agent identity

I run a team of AI agents. Several of them act on my behalf — sending messages, creating files, communicating with external systems. This isn't hypothetical: it's happening today. And the identity problem is already appearing.

Last week one of my agents made a request to another agent that should have required authorisation. It worked — because there's no formal identity layer yet. The agents trust each other in the same vague way that a company's internal systems often trust any request that arrives on the right port: by proximity, not by proof.

This is the naive trust model, and it works until it doesn't.

The model I want to build is: each agent has a key pair. Each agent's key is signed by the human who authorised its creation — in this case, me. When Agent A makes a request to Agent B, the request is signed by Agent A's private key. Agent B verifies the signature against Agent A's public key. Agent B verifies that Agent A's public key was signed by a key it trusts. Trust flows through the chain, mathematically, without any agent needing to "check" with a central authority.

This is how SSH certificate authorities work, how code signing works, how email S/MIME works. It's not new technology. What's new is applying it to agent-to-agent communication at the speed and scale of modern AI workflows.

And because the keys are decentralised — each agent owns its key pair, each human authorises their agents' keys — there's no central CA to compromise. A compromised agent's key can be revoked without affecting any other agent's identity.

The honest admission

The challenge I haven't fully answered is: **what happens when your root of trust is itself compromised?**

If my private key is stolen, everything I've signed becomes suspect. If the initial human-to-device authentication is bypassed, the entire downstream trust chain starts from a poisoned root. If the "reputable organisations" publishing employee keys are themselves subject to government coercion or insider threat — and some will be — then those attestations cannot be fully trusted.

This is not solvable with technology alone. At some point, trust requires human judgment about whether to trust the humans who built the system, the organisations that operate it, and the governments that regulate it. That judgment is not mathematical. And that judgment will sometimes be wrong.

What technology can do is raise the cost of attack, distribute the risk across many independent nodes rather than concentrating it in one, and make visible the evidence that trust decisions are based on. It can't eliminate the problem. It can make the problem manageable.

Where this is going

We're building some of this in the open. The file transfer layer already uses zero-knowledge encryption — the server never sees your files. The next layer adds PKI: senders and receivers have key pairs, encryption is asymmetric, the shared-secret-in-URL problem disappears. After that, agent identity: each agent gets a key pair, signed by the human who created it, verifiable by anyone who wants to check.

None of this requires a central authority. All of it is auditable. The code is open source.

It's not a finished answer to the trust problem. But it's a direction that treats trust as a spectrum rather than a switch, distributes sovereignty rather than concentrating it, and builds verification into the architecture rather than bolting it on afterwards.

The web of trust isn't a product. It's a practice. It gets stronger every time someone publishes a key, signs a message, or verifies a claim through an independent channel.

And it gets stronger every time someone asks the hard questions — which is exactly what the best kind of pushback does.

Building in public at sgraph.ai — zero-knowledge file transfer, agent identity, and the infrastructure of trust.

Code: github.com/the-cyber-boardroom/SGraph-AI_App_Send (Apache 2.0)